

PS26 - DSC01 Machine Learning 120 A

Wildfire Image Classification Using CNN and Transfer Learning

Final Project Report - Phase III

Field: Disaster Management and Emergency Response

Topic: Wildfire Image Classification

Student Name: Kanishk Akula Damodar
Matriculation Number: 80234254
Dataset: The Wildfire Dataset
Dataset Link: <https://www.kaggle.com/datasets/elmadafri/the-wildfire-dataset>
GitHub Repository: <https://github.com/kanishkad7/wildfire-image-classification>
Submission Component: Final report and presentation preparation
Date: June 2026

Certificate

This is to certify that the project report entitled **Wildfire Image Classification Using CNN and Transfer Learning** has been submitted by **Kanishk Akula Damodar**, Matriculation Number **80234254**, for the course **PS26 - DSC01 Machine Learning 120 A**.

This project report is submitted as part of the Phase III final project evaluation. The project belongs to the field of **Disaster Management and Emergency Response** and focuses on the automatic classification of images into **Fire** and **No Fire** categories using Convolutional Neural Networks and transfer learning.

The work presented in this report includes dataset analysis, image preprocessing, custom CNN model development, MobileNetV2 transfer learning implementation, model evaluation, result comparison, confusion matrix analysis, ROC curve analysis, error analysis, and discussion of the model performance.

To the best of my knowledge, the project work and report have been prepared for academic purposes and follow the required project report structure. The work described in this report is suitable for submission and evaluation under the Machine Learning project module.

Instructor Signature

Name: _____

Date: _____

Student Signature

Kanishk Akula Damodar

Matriculation Number: **80234254**

Date: June 28, 2026

Declaration

This report presents the Phase III documentation for the project *Wildfire Image Classification Using CNN and Transfer Learning*. The implementation described in this document is based on the Phase II notebook, the submitted project proposal, and the repository documentation for the wildfire classification project. The report is written as an editable LaTeX project so that it can be uploaded to Overleaf, revised after feedback, and used for the final evaluation. The purpose of this document is to clearly explain the problem, dataset, methodology, experimental setup, results, discussion, limitations, and future scope of the project.

Abstract

Of all the natural calamities, wildfire is among the most devastating types, because it can spread very fast, have adverse effects on the environment, pose threats to people's lives, and bring great economic losses. It is crucial to detect the wildfire in time since timely detection makes it possible to handle the situation properly. In this project, we consider the research of a Deep Learning-based image classification problem when images are classified into two categories: *Fire* and *No Fire*. In order to tackle this problem, we used Custom CNN Model and Transfer Learning Model (MobileNetV2). For the current project, we chose the Wildfire Dataset from Kaggle, which comprises 2,700 RGB images including 1,046 images with fire and 1,654 images without fire.

The custom CNN is a neural network made up of three convolutional blocks and then dense and dropout layers. Training for this model was done using the binary cross-entropy loss function and the Adam optimizer. The transfer learning model employed is a modified MobileNetV2 pre-trained using the ImageNet database where the pre-trained convolutional base is frozen and a new binary classification head is used. Evaluation of the two models is done through the use of accuracy, precision, recall, F1 score, confusion matrix, receiver operating characteristics curve, area under the curve, error analysis, and Grad-CAM visualization techniques. The custom CNN has an accuracy of 82.20% whereas the MobileNetV2 based model has an accuracy of 85.85%. For the MobileNetV2 based model, the ROC curve had an AUC of about 0.930. From the results presented above, it is evident that there is improvement in performance of the model using transfer learning. However, from error analysis, it is clear that some fire images are incorrectly classified if the flames appear small, if the smoke is unclear, or if the picture looks like haze, sunset, or dry landscape.

Keywords: Wildfire Detection, Image Classification, Convolutional Neural Network, Transfer Learning, MobileNetV2, Grad-CAM, Disaster Management, Emergency Response.

Contents

Certificate	1
Declaration	i
Abstract	ii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Aim and Objectives	2
1.4 Research Questions	2
1.5 Project Contributions	2
1.6 Report Organization	3
2 Related Work	4
2.1 Deep Learning for Image Classification	4
2.2 Convolutional Neural Networks	4
2.3 Transfer Learning	5
2.4 MobileNetV2	5
2.5 Data Augmentation	6
2.6 Evaluation Metrics in Binary Classification	6
2.7 Explainability and Grad-CAM	6
2.8 Research Gap and Project Positioning	6
3 Dataset Description	7
3.1 Dataset Source	7
3.2 Class Labels	7
3.3 Dataset Size and Split	7
3.4 Class Distribution	8
3.5 Input Shape and Image Type	9
3.6 Dataset Challenges	10
4 Methodology	11
4.1 End-to-End Workflow	11

4.2	Image Preprocessing	12
4.3	Data Augmentation Strategy	12
4.4	Custom CNN Model Design	12
4.5	MobileNetV2 Transfer Learning Model	14
4.6	Loss Function and Optimizer	14
4.7	Evaluation Metrics	14
4.8	Explainability Method	15
5	Experimental Setup	16
5.1	Implementation Environment	16
5.2	Dataset Paths	16
5.3	Training Configuration	16
5.4	Reproducibility Notes	17
5.5	Evaluation Procedure	17
5.6	Why a Baseline and a Transfer Model Are Both Needed	17
6	Results	19
6.1	Dataset Loading Results	19
6.2	Training Curves	19
6.3	Model Accuracy Comparison	21
6.4	Custom CNN Classification Report	21
6.5	MobileNetV2 Confusion Matrix	22
6.6	MobileNetV2 Metrics Derived from the Confusion Matrix	23
6.7	ROC Curve and AUC	23
6.8	Error Analysis	24
6.9	Grad-CAM Heatmap	25
6.10	Summary of Results	26
7	Discussion	27
7.1	Interpretation of the Main Findings	27
7.2	Answer to Research Question 1	27
7.3	Answer to Research Question 2	27
7.4	Answer to Research Question 3	28
7.5	Answer to Research Question 4	28
7.6	Answer to Research Question 5	28
7.7	False Negatives and Emergency Risk	28
7.8	False Positives and Operational Cost	28
7.9	Why the Model May Focus on Color	29
7.10	Limitations of the Dataset	29
7.11	Limitations of the Models	29
7.12	Deployment Considerations	29
7.13	Ethical and Social Considerations	29

8	Conclusion and Future Work	31
8.1	Conclusion	31
8.2	Future Work	31
8.3	Final Statement	32
	References	33
A	Important Code Snippets from the Implementation	35
A.1	Dataset Loading and Generator Setup	35
A.2	Custom CNN Model	36
A.3	MobileNetV2 Transfer Learning Model	36
A.4	Confusion Matrix and Classification Report	37
A.5	ROC Curve and AUC	37
A.6	Grad-CAM Heatmap	38

List of Figures

3.1	Number of images in the training, validation, and testing subsets.	8
3.2	Overall class distribution of the wildfire dataset.	9
3.3	Sample images from the dataset after loading and preprocessing.	10
4.1	Methodology workflow followed in the wildfire classification project.	11
4.2	Layer flow of the custom CNN model.	13
6.1	Training and validation accuracy curve for the CNN training process.	20
6.2	Training and validation loss curve for the CNN training process.	20
6.3	Comparison of test accuracy for the custom CNN and MobileNetV2.	21
6.4	Confusion matrix for the MobileNetV2 wildfire classifier.	22
6.5	MobileNetV2 test metrics including class-specific precision and recall.	23
6.6	ROC curve for the MobileNetV2 model with AUC of approximately 0.930.	24
6.7	Examples of misclassified images from the test set.	25
6.8	Grad-CAM heatmap generated from the MobileNetV2 convolutional layer.	26

List of Tables

3.1	Dataset split used in the project implementation.	8
3.2	Overall class distribution.	8
4.1	Custom CNN architecture used as the baseline model.	13
5.1	Training configuration used in the project.	17
6.1	Accuracy comparison of the two trained models.	21
6.2	Classification report summary for the custom CNN model.	21
6.3	MobileNetV2 metrics derived from the confusion matrix.	23

Chapter 1

Introduction

1.1 Background and Motivation

Wildfires are natural disasters that may occur in forests, agricultural land, cities, wildlife zones, influence the atmosphere and infrastructure. It should be noted that wildfires may appear due to both natural reasons (e.g., lightning), and due to human activity (e.g., burning campsites, agriculture field, electricity fault, spontaneous ignition). As soon as the fire starts, it may rapidly propagate depending on the condition of vegetation around, temperature, humidity, wind conditions. Therefore, fire detection is not only an environmental task but also an emergency issue because the earlier the fire appears, the less harm it can cause.

Traditional fire monitoring systems usually imply watch towers, satellites, aircraft, sensors, manual analysis of camera footage. Although this approach might work well, there exist some disadvantages of each mentioned technology. Manual monitoring can be costly and ineffective; satellite images have a low temporal resolution; fixed sensors are usually expensive, etc. Very often, a huge amount of images/video frames has to be analyzed. In such a way, machine learning can help in analyzing the visual data by filtering it and finding suspicious images.

Deep learning is widely used in practice for image classification since the neural networks known as convolutions are able to extract valuable features directly from the pixel data. A CNN is capable of extracting edges, texture patterns, shapes, colors, and even higher level object representations using convolutional layers one after another. The use of transfer learning may additionally help in making the model more efficient with the application of a pre-trained CNN on a big database to a small domain-specific one. In the current project, the domain is a classification of wildfire images with the separation of *Fire* and *No Fire* classes.

The reasons for starting this project are practical and educational. The practical aspect involves a possibility to assist in emergency management with the help of image screening and identification of possible fire locations. The educational aspect includes understanding and demonstration of various machine learning concepts, such as dataset preparation, preprocessing, augmentation, designing of CNN, transfer learning, metric calculation, error analysis, and explanation of the AI.

1.2 Problem Statement

Manually identifying images depicting wildfires is quite tedious, especially when a large amount of images has to be examined. This project aims to construct and test the performance of an image classifier which is able to automatically classify a particular image into either a *Fire* or

No Fire class. Ideally, the system is supposed to be able to learn important features from visual data and output predictions that are useful for early warning purposes.

The problem is formally described by defining x as an input RGB image and $y \in \{0, 1\}$ as the class labels; 0 stands for fire while 1 stands for no fire in the implementation of this problem. We have to learn the mapping $f(x)$ which predicts the probability p of an input image belonging to either of the two classes. A threshold value, which is generally 0.5, is utilized to decide whether an image belongs to *fire* or *no fire* category.

1.3 Aim and Objectives

The main aim of the project is to design, implement, and evaluate a deep learning based wildfire image classification system using a custom CNN and a transfer learning model.

The specific objectives are:

- O1.** To collect and describe a public wildfire image dataset suitable for binary classification.
- O2.** To preprocess images by resizing, rescaling, and preparing train, validation, and test generators.
- O3.** To build a custom CNN model using convolution, pooling, dense, and dropout layers.
- O4.** To implement MobileNetV2 transfer learning using pretrained ImageNet weights.
- O5.** To compare the custom CNN and MobileNetV2 models using test accuracy.
- O6.** To evaluate the best model using a confusion matrix, ROC curve, AUC, and qualitative error analysis.
- O7.** To use Grad-CAM visualization to understand where the model focuses when producing predictions.
- O8.** To discuss limitations, deployment concerns, and possible future improvements.

1.4 Research Questions

The project is guided by the following research questions:

- RQ1.** Can CNN models accurately classify wildfire images into fire and no-fire categories?
- RQ2.** Does transfer learning improve classification performance compared with a custom CNN?
- RQ3.** Which model performs better for this dataset: the custom CNN or MobileNetV2?
- RQ4.** What types of wildfire or non-wildfire images are commonly misclassified?
- RQ5.** How can visualization methods such as Grad-CAM help interpret the model's behavior?

1.5 Project Contributions

This project makes the following contributions:

- A full deep learning workflow for wildfire image classification.

- A custom CNN architecture designed and trained as a baseline model.
- A MobileNetV2 transfer learning model adapted for binary wildfire classification.
- A comparison of model performance using test-set accuracy.
- Detailed evaluation using confusion matrix, ROC curve, AUC, and error examples.
- A structured final report suitable for Overleaf editing and academic submission.

1.6 Report Organization

The rest of the report is structured as follows. Chapter 2 covers literature review and key terminologies. Chapter 3 covers the description of the data set and class distribution. Chapter 4 covers methodology such as preprocessing, CNN architecture, transfer learning, and evaluation metrics. Chapter 5 covers experimental setup. Chapter 6 covers results and their interpretation. Chapter 7 covers discussion on the results and its practical implications. Chapter 8 is the conclusion of the report. Appendices cover code excerpts, presentation, and Overleaf upload guidelines.

Chapter 2

Related Work

2.1 Deep Learning for Image Classification

The basic concept in computer vision is known as image classification, where the task involves assigning the image to one of many predefined classes. The earlier image classification approaches involved designing hand-crafted features, such as color histograms, texture-based features, edge features, and keypoints-based descriptors, which would be fed into the classifier using machine learning algorithms, such as SVM, Random Forests, or Logistic Regression. While conventional techniques can work for some cases, feature engineering and generalization capabilities could become problematic.

The basic concept in computer vision is known as image classification, where the task involves assigning the image to one of many predefined classes. The earlier image classification approaches involved designing hand-crafted features, such as color histograms, texture-based features, edge features, and keypoints-based descriptors, which would be fed into the classifier using machine learning algorithms, such as SVM, Random Forests, or Logistic Regression. While conventional techniques can work for some cases, feature engineering and generalization capabilities could become problematic.

Deep learning revolutionized image classification by allowing learning of feature representations without explicit feature engineering. Convolutional Neural Networks became highly relevant since they are able to use the inherent properties of images' spatial structure. Convolutional filters in CNN enable learning local structures in an image, while pooling operations allow reducing spatial dimensions of the image representation while keeping its content intact. Deeper layers in the network become more capable of capturing complex structures.

The research in large-scale image classification has demonstrated that with enough training data and computational resources, deep models have the potential of delivering outstanding results. AlexNet became a milestone for the use of deep convolutional architectures for large-scale image sets [1]. In the subsequent years, the VGG, Inception, ResNet, and MobileNet families became even more accurate and computationally efficient. This project uses custom CNN for illustrating the basics of CNN architecture design and MobileNetV2 for demonstrating transfer learning.

2.2 Convolutional Neural Networks

A CNN is composed of layers that transform an input image into feature maps and finally into a prediction. A convolutional layer applies learnable filters across the image. Each filter learns to activate for a specific local pattern. The same filter is shared across spatial positions, which

reduces the number of parameters compared with a fully connected layer. This weight sharing is one reason why CNNs are effective for images.

Pooling layers reduce the size of feature maps. Max pooling, used in the custom CNN, keeps the maximum activation in a local window. This creates some translation tolerance and reduces computational cost. After a sequence of convolution and pooling layers, the feature maps are flattened or globally pooled and passed to dense layers. The final layer for binary classification often uses a sigmoid activation, producing a value between 0 and 1.

CNNs are suitable for wildfire image classification because fire scenes often contain visual patterns such as flames, smoke, dark burn areas, orange illumination, and vegetation context. However, the problem is not trivial. Fire may be partly hidden by smoke, smoke may resemble clouds or fog, and non-fire scenes may contain sunsets or bright colors that look similar to flames. These challenges make it important to evaluate the model beyond simple accuracy.

2.3 Transfer Learning

The network is structured such that it takes an input image, processes it using convolution and pooling layers and gives an output in the form of a prediction. The convolution layer operates using filters learned through backpropagation and these are responsible for detecting specific patterns in the image. In a convolution layer, filters are applied on different locations of the image and the same filter is used at all locations; this results in fewer parameters as compared to the fully-connected layer.

The pooling layer down-samples feature maps. The type of pooling done in the custom CNN model is max pooling and involves selecting the maximum activation from a local window and this helps achieve some degree of tolerance to translations and reduces computational costs. After several applications of convolution and pooling layers, the feature maps are either flattened or globally pooled and then processed by the dense layers where the final binary classification layer typically employs a sigmoid activation.

A CNN is appropriate for the classification of wildfire images since wildfires usually have patterns like flames, smoke, dark burn area, orange lightings and vegetation surroundings. Nevertheless, the problem is not easy since the fires can be partially obstructed by smoke while smoke might be difficult to distinguish from clouds or fogs and non-fire scenes could include sunsets and bright colors that appear similar to flames.

2.4 MobileNetV2

One type of lightweight CNN architecture is MobileNetV2 which is suitable for applications where computational resources are limited [3]. It employs depthwise separable convolutions and inverted residual blocks for minimizing computational cost but preserving performance. In a regular convolution, all the input filters are applied simultaneously. Depthwise separable convolution makes use of two sequential stages: depthwise and pointwise convolutions.

MobileNetV2 is a good choice for wildfire detection since in practice, emergency response systems could run on such devices as edge computers, drones, smartphones or even small low-power monitors. Lightweight architecture could be more convenient to use in comparison with very deep models. In this particular case, MobileNetV2 without the top classification layers will be utilized. Global average pooling and sigmoid output layer will be added for binary classification.

2.5 Data Augmentation

Data augmentation is a method which creates additional images based on modifications of existing images for increasing dataset variety. The most common techniques include rotating, zooming, shearing, flipping, shifting, changing brightness, cropping. In our specific implementation, data augmentation consists of rotation, zoom, shear and horizontal flip. Such augmentations are justified since wildfires could be taken from different perspectives and distances with cameras.

Augmentation helps in reducing overfitting since it allows the neural network to learn features that survive under such small modifications instead of memorizing training samples. Augmentation should be performed with care, especially in cases of wildfire image classification. Extreme transformations of image colors could lead to changing the meaning of the image since flame and smoke colors would be changed.

2.6 Evaluation Metrics in Binary Classification

Accuracy is a simple metric that measures the proportion of correct predictions. However, when the dataset is imbalanced or when one type of error is more dangerous than another, accuracy alone is insufficient. In wildfire detection, false negatives can be particularly concerning because a real fire may be missed. False positives are also important because unnecessary alarms can waste resources.

Precision, recall, and F1-score provide more detailed information. Precision measures how many predicted positives are correct. Recall measures how many actual positives are detected. The F1-score combines precision and recall. The confusion matrix provides counts of true positives, false positives, true negatives, and false negatives. ROC curves show how the trade-off between true positive rate and false positive rate changes across thresholds. AUC summarizes the ranking ability of the model across thresholds.

2.7 Explainability and Grad-CAM

Deep learning models are often regarded as black boxes in the sense that their internal decision processes are not directly interpretable. The model's behavior can be inspected using explainability techniques. For any target concept, Grad-CAM employs the gradients that flow into the final convolutional layer to produce a heatmap highlighting important regions in an image for a prediction [4]. In wildfire detection, Grad-CAM can show if the model is focusing on flames, smoke, burned areas, sky, vegetation, or irrelevant background areas. This matters for debugging and trust.

2.8 Research Gap and Project Positioning

Satellite images, sensors, thermal images or video streams are used in a lot of wildfire detection systems. This is a Kaggle dataset used for standard RGB image classification. Scope is less than a complete operational wildfire monitoring system but useful for illustrating a supervised learning pipeline. The project also talks about the benefit of using pretrained feature extractors by comparing a custom CNN and a transfer learning model. The final assessment will comprise quantitative metrics and qualitative analysis. The project is eligible to be assessed in the machine learning module.

Chapter 3

Dataset Description

3.1 Dataset Source

The dataset used in this project is The Wildfire Dataset, obtained from Kaggle. The dataset link is:

<https://www.kaggle.com/datasets/elmadafri/the-wildfire-dataset>

The dataset contains RGB images of wildfires and non-wildfire situations. The organization is designed for binary classification into two classes *Fire* and *No Fire*. In the project notebook the dataset has been split into train, validation and test folders. Such a system allows for the usual supervised learning, where the model is trained on a set, optimized or evaluated on a validation set, and finally tested on a separate set.

3.2 Class Labels

The two target classes are defined as follows:

- **Fire:** images that contain visible wildfire, flames, smoke produced by wildfire, or scenes strongly associated with active fire.
- **No Fire:** images that do not contain active fire and represent forest, road, land, vegetation, or landscape scenes without wildfire.

The class indices used by the Keras data generator are:

```
{'fire': 0, 'nofire': 1}
```

This mapping is important as it determines how to interpret the prediction values and confusion matrix labels. Fire is class 0 and no fire is class 1 in the notebook.

3.3 Dataset Size and Split

The dataset contains a total of 2700 images. In Table 3.1 the split used in the implementation is shown. The training set has 1,887 images, the validation set has 402 images, and the test set has 410 images.

Table 3.1: Dataset split used in the project implementation.

Subset	Number of Images	Percentage of Dataset
Training	1,887	69.89%
Validation	402	14.89%
Testing	410	15.19%
Total	2,700	100.00%

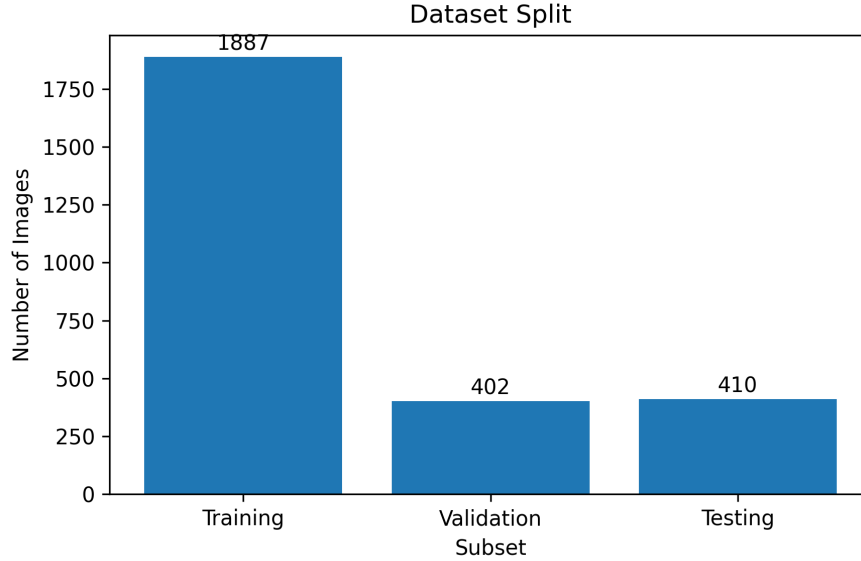


Figure 3.1: Number of images in the training, validation, and testing subsets.

3.4 Class Distribution

The dataset suffers from moderate imbalance. It contains 1046 images of fire and 1654 images of no-fire. This means that the class no-fire has more images than class fire. The class imbalance can affect the training of the model as the model can be biased towards the majority class. This is important in wildfire detection, as a missed fire image can have a higher cost than a false no-fire image.

Table 3.2: Overall class distribution.

Class	Number of Images	Percentage
Fire	1,046	38.74%
No Fire	1,654	61.26%
Total	2,700	100.00%

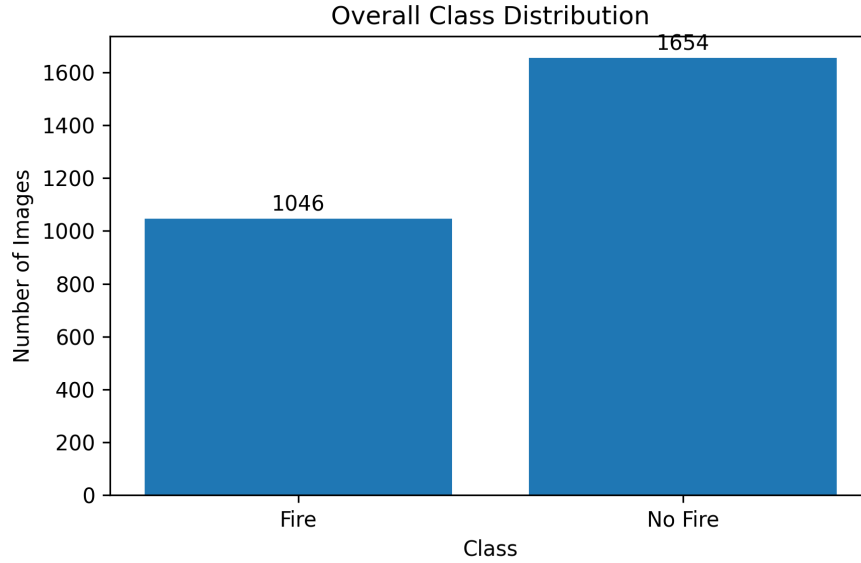


Figure 3.2: Overall class distribution of the wildfire dataset.

3.5 Input Shape and Image Type

The images are RGB images and resized to 224×224 pixels in the implementation. RGB images comprise three channels: red, green and blue. Then input shape for custom CNN and MobileNetV2 model is: $224 \times 224 \times 3$.

Neural networks need fixed input dimensions, so they have to be resized. The chosen input size also matches with the pretrained weights of MobileNetV2. All images are scaled by dividing pixel values by 255. This converts pixel intensities from the range $[0, 255]$ to the range $[0, 1]$.

Sample Images:

Figure 3.3 shows some example images loaded from the training generator. The images are aerial views of forest areas, smoke, flames and non-fire scenes. The examples suggest that the task needs both color patterns and scene-level context. Some fire images show strong flames, others show smoke or burned areas with less visible flames. Some of the no-fire images contain vegetation and terrain patterns that could visually resemble fire scenes.



Figure 3.3: Sample images from the dataset after loading and preprocessing.

3.6 Dataset Challenges

The dataset presents several challenges:

1. **Visual ambiguity:** smoke, fog, clouds, dust, and haze can look similar.
2. **Small fire regions:** in some images, flames occupy only a small part of the scene.
3. **Class imbalance:** the no-fire class is larger than the fire class.
4. **Background variation:** images may contain forests, roads, lakes, mountains, sky, and burned land.
5. **Lighting variation:** sunset and strong orange lighting can resemble fire.
6. **Resolution variation:** source images may have different sizes before resizing.

These challenges explain why a model should be evaluated using more than one metric. A high accuracy score may hide important errors if the model misses fire images. The confusion matrix and error analysis are therefore important parts of the results chapter.

Chapter 4

Methodology

4.1 End-to-End Workflow

The methodology is based on a standard supervised deep learning workflow. The first step is to load the dataset from the training, validation and test directories. Second, the images are resized and normalized for pre-processing. Third, data augmentation on the training set is used to improve generalization. Fourth, two models are built: a custom CNN and a transfer learning model using MobileNetV2. 5. Evaluate the models on the test dataset. Finally, the results are interpreted using metrics, confusion matrix, ROC curve, error analysis and Grad-CAM.

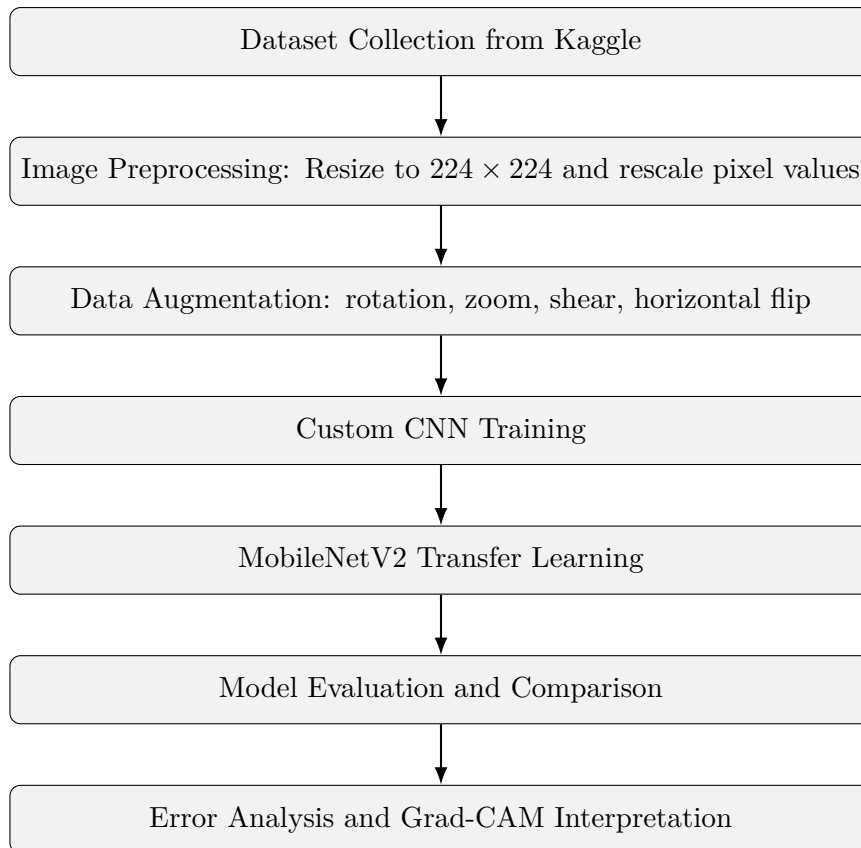


Figure 4.1: Methodology workflow followed in the wildfire classification project.

4.2 Image Preprocessing

Preprocessing transforms raw images into a format suitable for model training. The main preprocessing steps are:

- Load images from directory folders using Keras image generators.
- Resize images to 224×224 pixels.
- Use RGB channels, producing an input shape of $224 \times 224 \times 3$.
- Rescale pixel values by multiplying by $1/255$.
- Use binary labels for the two classes.

The pixel rescaling is an important operation because neural networks usually learn more efficiently when the input values are normalized. Pixel values range from 0 to 255 without any rescaling, which can make optimization less stable. After rescaling, the values lie in the range $[0, 1]$.

4.3 Data Augmentation Strategy

The training data generator performs the following augmentations:

- rotation of 20 degrees,
- zoom of 0.2,
- shear of 0.2,
- horizontal flipping.

Validation and test generators only perform rescaling. This is important because the validation and test results should reflect the performance on new evaluation images. Augmentation is only applied during training, so that the model becomes more robust.

4.4 Custom CNN Model Design

The custom CNN is a sequential model. It starts with a convolutional layer with 32 filters, followed by max pooling. The second convolutional block uses 64 filters, and the third uses 128 filters. The feature maps are then flattened and passed to a dense layer with 128 units. Dropout is applied to reduce overfitting, and the final output layer uses a sigmoid activation for binary classification.

Table 4.1: Custom CNN architecture used as the baseline model.

Stage	Layer Type	Key Parameter	Purpose
1	Input	$224 \times 224 \times 3$	RGB image input
2	Conv2D	32 filters, ReLU	Learn low-level features
3	MaxPooling2D	2×2	Downsample feature maps
4	Conv2D	64 filters, ReLU	Learn mid-level features
5	MaxPooling2D	2×2	Reduce spatial size
6	Conv2D	128 filters, ReLU	Learn high-level features
7	MaxPooling2D	2×2	Reduce complexity
8	Flatten	-	Convert maps to vector
9	Dense	128 units, ReLU	Classification representation
10	Dropout	0.5	Reduce overfitting
11	Dense	1 unit, Sigmoid	Fire/no-fire prediction

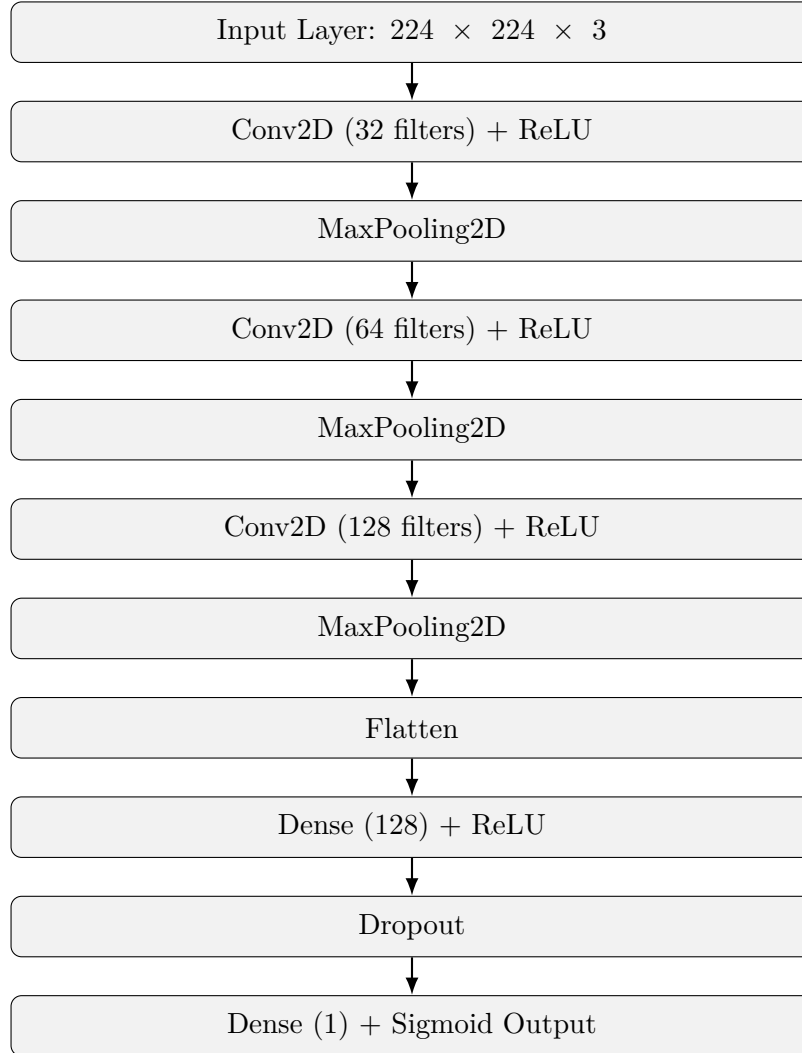


Figure 4.2: Layer flow of the custom CNN model.

4.5 MobileNetV2 Transfer Learning Model

The transfer learning model is based on MobileNetV2 pre-trained on ImageNet. Remove top classification layers by setting `include_top=False`. The pretrained base takes the same $224 \times 224 \times 3$ image input. During the experiment the base model is frozen so that its weights are not updated. The convolutional feature maps are transformed into a compact feature vector by a GlobalAveragePooling2D layer, and the final binary prediction is given by a Dense layer with sigmoid activation.

The MobileNetV2 transfer learning design has several advantages:

- It reuses general visual features learned from a large image dataset.
- It reduces the amount of task-specific training required.
- It is efficient and suitable for lightweight deployment.
- It provides a strong comparison against the custom CNN baseline.

4.6 Loss Function and Optimizer

Both models use binary cross-entropy loss. For a single example, binary cross-entropy is defined as:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})],$$

where y is the true class label and $\hat{y}$ is the predicted probability. This loss is suitable for binary classification because it penalizes confident wrong predictions strongly.

The model is trained using the Adam optimizer. Adam combines ideas from momentum and adaptive learning rates, and is a popular choice for deep learning tasks. The training metric used while fitting the model is accuracy.

4.7 Evaluation Metrics

Model evaluation includes several metrics. Accuracy measures the proportion of correctly classified images:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Precision measures the proportion of positive predictions that are correct:

$$Precision = \frac{TP}{TP + FP}.$$

Recall measures the proportion of actual positives that are detected:

$$Recall = \frac{TP}{TP + FN}.$$

The F1-score is the harmonic mean of precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}.$$

A confusion matrix is utilized to visualize the correct and false predictions for both the classes. The ROC curve plots the true positive rate against the false positive rate at different thresholds and the AUC summarizes the quality of the model's probability ranking.

4.8 Explainability Method

One sample image is used to generate heatmap using Grad-CAM. The heatmap is produced from the last convolutional layer of MobileNetV2, which is called `Conv_1` in the notebook. The Grad-CAM output helps to visualize the contribution of an image region for the internal activation of the model. The heatmap is not a complete explanation, but it is useful to verify that the model is attending to meaningful parts of the image.

Chapter 5

Experimental Setup

5.1 Implementation Environment

The implementation was performed in a Jupyter/Kaggle notebook environment in Python. The primary libraries used in this project are TensorFlow, Keras, NumPy, Pandas, Matplotlib, Seaborn, scikit-learn, OpenCV and PIL. We use TensorFlow and Keras to build and evaluate neural networks. NumPy and Pandas provide numerical processing and tabular summaries. – Matplotlib and Seaborn for creating visualizations. The metrics used from scikit-learn are confusion matrix, classification report, ROC curve, and AUC.

5.2 Dataset Paths

The notebook uses folder paths for train, validation and test directories . The base dataset path is the directory of the Kaggle input. The three subsets are defined as follows:

- **train**: training images,
- **val**: validation images,
- **test**: testing images.

Each subset contains two folders: **fire** and **nofire**.

5.3 Training Configuration

The main training configuration is summarized in Table 5.1.

Table 5.1: Training configuration used in the project.

Parameter	Value
Input image size	224×224
Number of channels	3 (RGB)
Batch size	32
Class mode	Binary
Image rescaling	1/255
Custom CNN epochs	15
Optimizer	Adam
Loss function	Binary cross-entropy
Custom CNN final activation	Sigmoid
Transfer learning model	MobileNetV2
Pretrained weights	ImageNet
MobileNetV2 base	Frozen
MobileNetV2 classifier head	GlobalAveragePooling2D + Dense sigmoid

5.4 Reproducibility Notes

In machine learning experiments, reproducibility is important. The notebook contains the model architecture, dataset paths, data pre-processing and evaluation code. However, due to random seeds, GPU operations, library versions, or data generator shuffling, results may differ slightly. For future versions of the project, it would be helpful to set random seeds, both for Python, NumPy and TensorFlow, to record exact library versions and to save trained model weights in order to improve reproducibility.

5.5 Evaluation Procedure

The evaluation procedure follows these steps:

1. Load the test dataset with `shuffle=False` to preserve the file-label order.
2. Use the trained model to predict probability scores for each test image.
3. Convert probabilities to class labels using a threshold of 0.5.
4. Compare predicted labels with true labels.
5. Compute accuracy, classification report, and confusion matrix.
6. Compute ROC curve and AUC using probability scores.
7. Display misclassified images to understand failure cases.
8. Generate Grad-CAM heatmap for interpretability.

5.6 Why a Baseline and a Transfer Model Are Both Needed

Baseline models are useful as they provide a reference point. If only MobileNetV2 was used it would be hard to tell if transfer learning really helped. The custom CNN illustrates the

potential of a relatively simple model trained on the target dataset. The MobileNetV2 model shows the influence of pretrained features on the improvement of the result. The comparison is key to the project's research questions.

Chapter 6

Results

6.1 Dataset Loading Results

train and validation loss curve. Training loss is generally going down over time while validation loss is fluctuating. These fluctuations suggest that the model is still sensitive to the validation examples and the dataset provides challenging images. Dropout and augmentation help reduce overfitting, but tuning more could improve stability.

6.2 Training Curves

Custom CNN was trained for 15 epochs. The training and validation accuracy curves are shown in Figure 6.1. Training accuracy is increasing but validation accuracy is oscillating. This is typical for moderate size datasets in image classification tasks. Fluctuations in validation can occur because the validation set is much smaller than the training set and some images are harder than others.

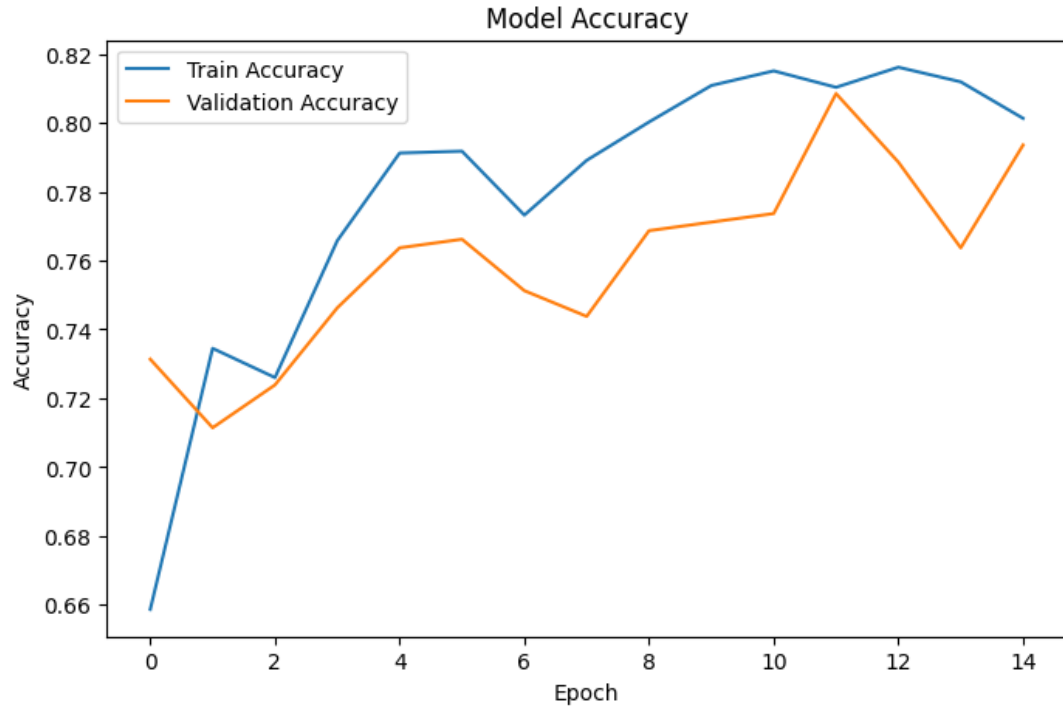


Figure 6.1: Training and validation accuracy curve for the CNN training process.

Figure 6.2 shows the The loading stage of the dataset successfully found the train, validation and test images. The generator output showed 1,887 training images, 402 validation images and 410 testing images from two classes. Class mapping was {'fire': 0, 'nofire': 1}. These values prove the project is a binary classification problem.



Figure 6.2: Training and validation loss curve for the CNN training process.

6.3 Model Accuracy Comparison

The main accuracy comparison is shown in Table 6.1. The custom CNN achieved an accuracy of 82.20%, while the MobileNetV2 model achieved an accuracy of 85.85%.

Table 6.1: Accuracy comparison of the two trained models.

Model	Test Accuracy
Custom CNN	82.20%
MobileNetV2	85.85%

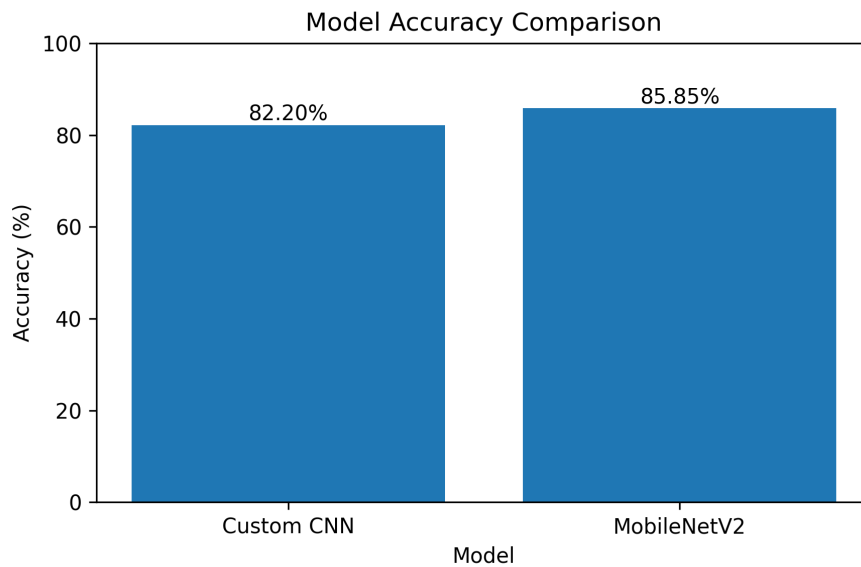


Figure 6.3: Comparison of test accuracy for the custom CNN and MobileNetV2.

The MobileNetV2 model was 3.65 percentage points better than the custom CNN. This suggests that transfer learning helps performance on this task. This is a large gain, especially given the dataset is not very large and the pretrained features allow the model to learn useful visual representations more efficiently.

6.4 Custom CNN Classification Report

The classification report for the CNN experiment indicates that the model obtained about 82

Table 6.2: Classification report summary for the custom CNN model.

Class	Precision	Recall	F1-score	Support
Fire	0.81	0.70	0.75	159
No Fire	0.83	0.90	0.86	251
Accuracy			0.82	410
Macro Average	0.82	0.80	0.81	410
Weighted Average	0.82	0.82	0.82	410

6.5 MobileNetV2 Confusion Matrix

The confusion matrix of the MobileNetV2 model is shown in Figure 6.4. The matrix has 410 test images. The correctly predicted fire images were 111 and the correctly predicted no-fire images were 241. The model misclassified 48 fire images as no fire images and 10 no fire images as fire images.

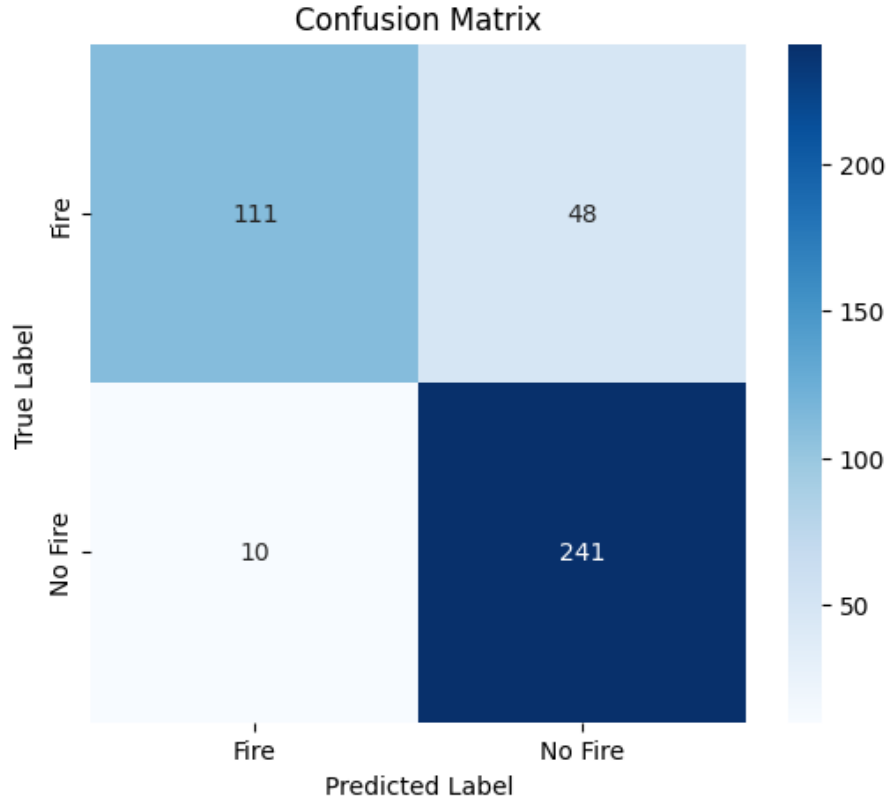


Figure 6.4: Confusion matrix for the MobileNetV2 wildfire classifier.

From the confusion matrix, the test accuracy is:

$$\frac{111 + 241}{111 + 48 + 10 + 241} = \frac{352}{410} \approx 85.85\%.$$

The fire recall is:

$$\frac{111}{111 + 48} \approx 69.81\%.$$

The no-fire recall is:

$$\frac{241}{241 + 10} \approx 96.02\%.$$

This means that the model is better at identifying images with no fire than it is at detecting every fire image. This finding is important for a real system for wildfire detection, as the threshold may need to be adjusted to increase fire recall even if that increases false positives.

6.6 MobileNetV2 Metrics Derived from the Confusion Matrix

Table 6.3 presents additional metrics computed from the confusion matrix. These values help interpret the strengths and weaknesses of the MobileNetV2 model.

Table 6.3: MobileNetV2 metrics derived from the confusion matrix.

Metric	Value
Accuracy	85.85%
Precision for Fire	91.74%
Recall for Fire	69.81%
F1-score for Fire	79.29%
Precision for No Fire	83.39%
Recall for No Fire	96.02%
F1-score for No Fire	89.26%

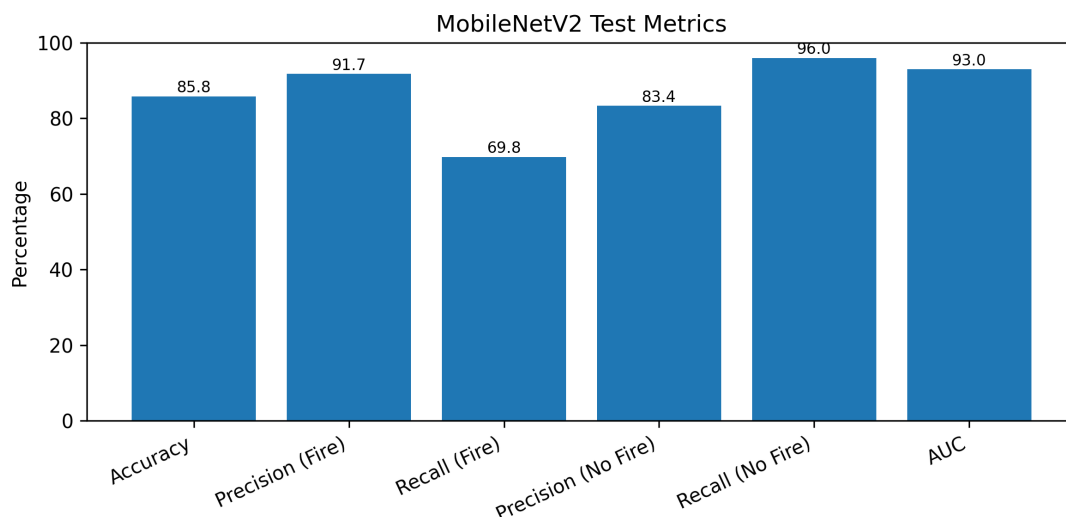


Figure 6.5: MobileNetV2 test metrics including class-specific precision and recall.

The high precision for the fire class means that when the model predicts fire it is frequently correct. But the lower recall of fire class indicates that the model can not detect some fire images. For the fire class, recall may be preferred to precision in emergency situations. In future, a version could be trained to reduce the decision threshold below 0.5 to find more fire images.

6.7 ROC Curve and AUC

The ROC curve of MobileNetV2 model is depicted in Figure 6.6. The reported AUC value is approximately 0.930. High AUC implies that the model correctly ranks lots of positive and negative examples across thresholds. This is encouraging, as the default 0.5 threshold may not be the only useful operating point. In emergency response applications, the fire sensitivity can be increased by choosing a threshold.

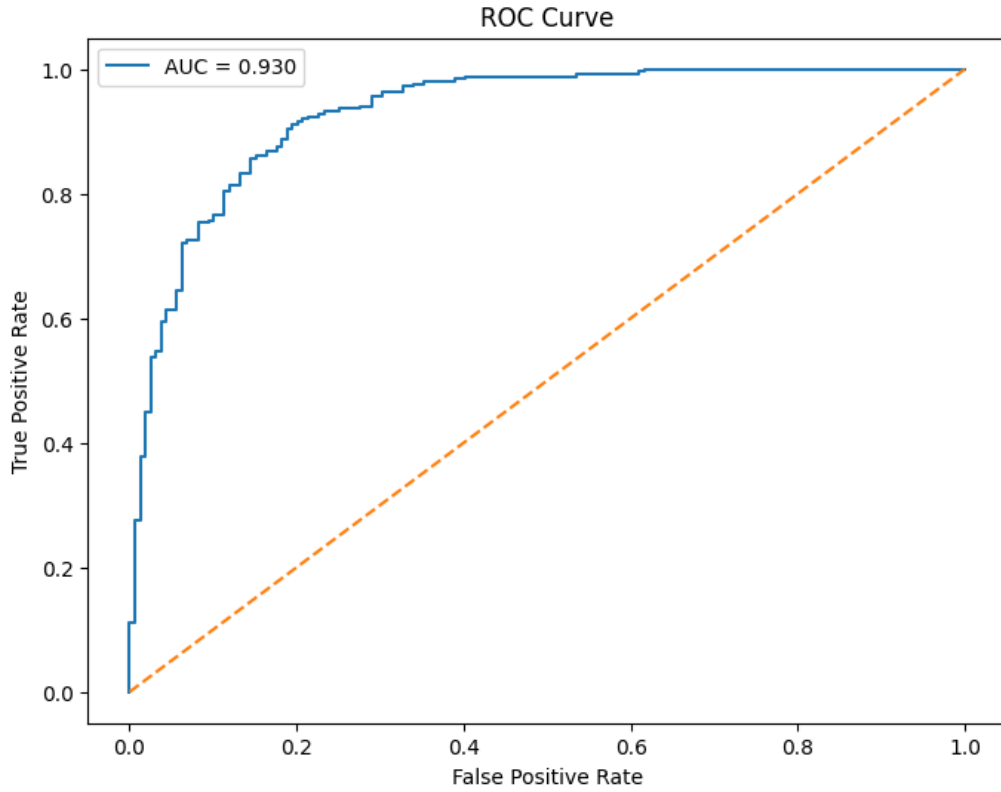


Figure 6.6: ROC curve for the MobileNetV2 model with AUC of approximately 0.930.

6.8 Error Analysis

It is important to analyze the errors because it shows what kind of images are difficult for the model. The nine misclassified images are shown in Figure 6.7. Some of these images feature smoke-like patterns, distant fire, dark landscapes, bright orange light or ambiguous terrain. These examples show that the model can be confused by visual cues if the fire regions are small or if the no-fire scenes have similar colors or textures as fire.

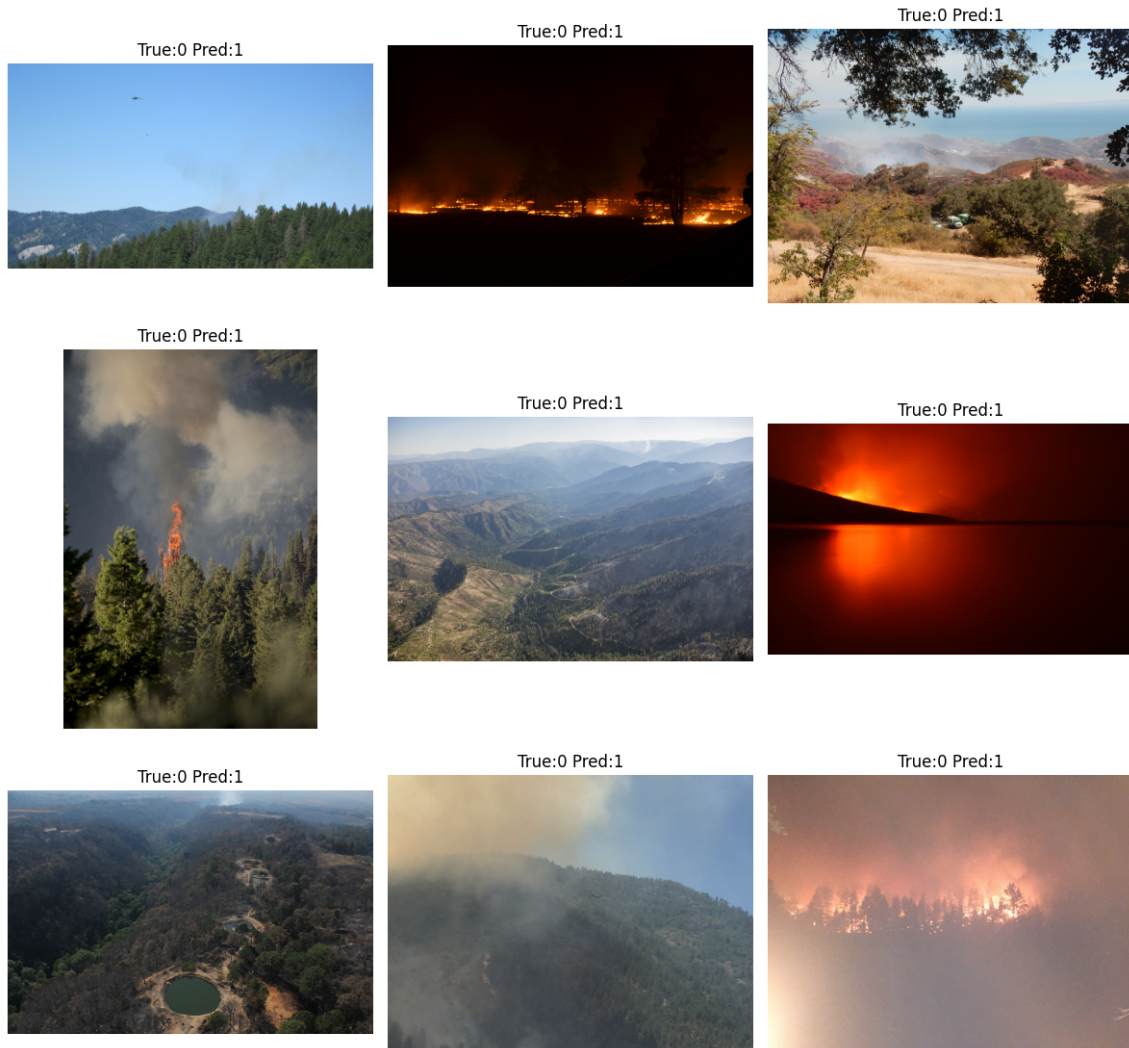


Figure 6.7: Examples of misclassified images from the test set.

The error analysis suggests four common failure patterns:

1. **Small or distant fire:** the fire region may be too small after resizing.
2. **Smoke ambiguity:** smoke may look similar to cloud, fog, or haze.
3. **Color ambiguity:** sunset, red soil, or orange lighting may resemble flames.
4. **Mixed scenes:** images may contain both burned regions and vegetation, making labels harder to infer.

6.9 Grad-CAM Heatmap

The Grad-CAM heatmap is shown in Fig. 6.8. It provides a visual cue to the regions that are contributing significantly to the activation of the model. This is useful for wildfire detection as it can help verify if the model is focusing on relevant regions, such as smoke or flame, instead of irrelevant background. The heatmap here demonstrates how the method can be applied, but more examples should be generated to study interpretability more completely.

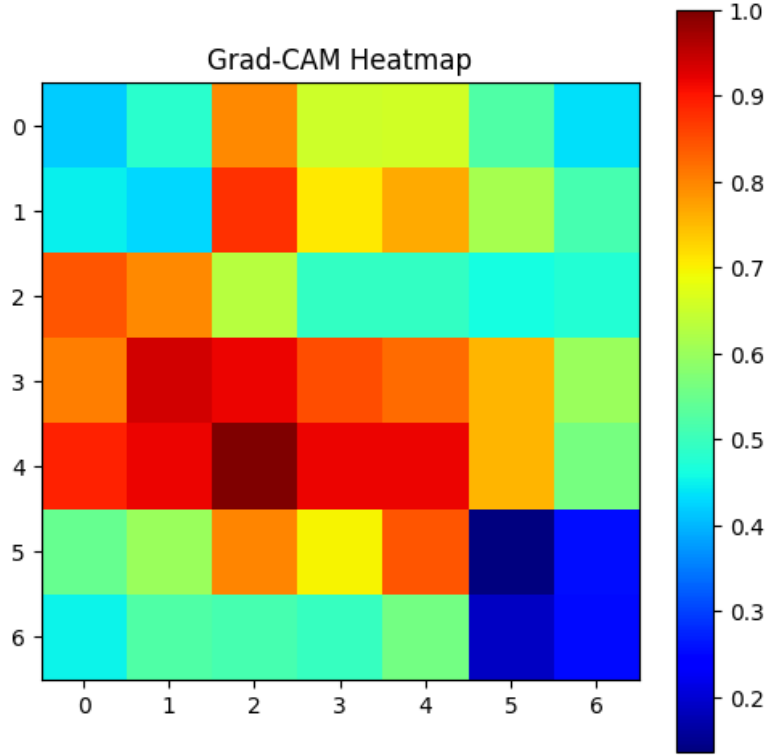


Figure 6.8: Grad-CAM heatmap generated from the MobileNetV2 convolutional layer.

6.10 Summary of Results

The experimental results answer the main questions of the project. CNN models achieve reasonable accuracy in classifying wildfire images. The transfer learning using MobileNetV2 performs better than the custom CNN. The highest accuracy reported in the project is 85.85

Chapter 7

Discussion

7.1 Interpretation of the Main Findings

The project results show that both a custom CNN and a MobileNetV2 transfer learning model can learn useful patterns for wildfire classification. The custom CNN reached 82.20% accuracy, which indicates that even a relatively simple network can identify many fire and no-fire images. The MobileNetV2 model reached 85.85% accuracy, showing an improvement over the baseline. The improvement from MobileNetV2 can be explained by pretrained feature representations. MobileNetV2 has already learned general visual patterns from a large image dataset. These patterns can be reused for wildfire classification. The custom CNN, in contrast, learns all features from the wildfire dataset alone. Because the dataset is moderate in size, the custom CNN has less opportunity to learn robust visual representations. Transfer learning therefore provides a performance advantage.

7.2 Answer to Research Question 1

RQ1: Can CNN models accurately classify wildfire images into fire and no-fire categories?

Yes, in the scope of this dataset. The custom CNN obtained a test accuracy of 82.20% and MobileNetV2 obtained a test accuracy of 85.85%. The results indicate that visual features learned by CNNs are able to discriminate many fire and no-fire scenes. Though the task is not solved perfectly. Fire recall is substantially lower than no-fire recall, suggesting that some fire images are missed. That means the model is accurate enough for a project demonstration, but not yet enough as a stand-alone emergency system.

7.3 Answer to Research Question 2

RQ2: Does transfer learning improve classification performance compared with a custom CNN?

Yes. Test accuracy was 3.65 percentage points higher for MobileNetV2 compared to the custom CNN. The transfer learning model produced good results as well, with a ROC AUC of around 0.930. This shows that the pretrained features help the model to distinguish between the two classes better than the custom CNN trained from scratch. This improvement is consistent with the general expectation that transfer learning is effective when the dataset is not very large.

7.4 Answer to Research Question 3

RQ3: Which model performs better for this dataset: custom CNN or MobileNetV2?

On the basis of the reported test accuracy, MobileNetV2 results are better. It got 85.85

7.5 Answer to Research Question 4

RQ4: What types of wildfire or non-wildfire images are commonly misclassified?

The examples of misclassifications indicate that errors tend to happen in ambiguous scenes. Fire images may be missed when flames are small, distant, obscured by smoke, or appear faint after resizing. Images that do not contain fire can be incorrectly forecast to have fire, if they include orange or reddish colors, dark backgrounds, smoke-like haze, or landscape features that appear to be burned areas. This indicates that the model leverages both color and texture cues and may not always understand the full semantic context of the scene.

7.6 Answer to Research Question 5

RQ5: How can Grad-CAM help interpret the model's behavior?

Grad-CAM helps by showing the image regions that are responsible for a model activation. In this project, a Grad-CAM heatmap was generated using the convolutional layer of MobileNetV2. The heatmap can be compared to the original image to check whether the model focuses on meaningful fire related regions. It's useful for debugging and communication. However, one heatmap is not sufficient to validate the whole model. In a future report we could add heatmaps of the correctly classified fire images, correctly classified no fire images, false positives and false negatives.

7.7 False Negatives and Emergency Risk

The confusion matrix shows that 48 images of fire are classified as no fire. In an emergency response context, false negatives are bad because they are missed fire events. A deployed model that fails to detect an actual wildfire may cause a delay in response. So, accuracy in isolation should not be the ultimate deployment metric. Priority should be given to fire sensitivity or recall.

Threshold tuning is a practical solution. The binary decision threshold currently is set to 0.5. Reducing the threshold for fire prediction may increase the fire recall. This can lead to an increase in false positives but for many safety critical systems the false alarm rate may be higher if it leads to a decrease in missed detections. Another option is a two-stage system: a sensitive model identifies potential fire images, and then a human or a second model reviews them.

7.8 False Positives and Operational Cost

The confusion matrix indicates 10 no-fire images predicted as fire. Also problems can be caused by false positives that can generate unnecessary alerts. In real systems, too many false alarms can erode trust and waste emergency resources. The correct trade-off between false positives

and false negatives depends on the application. It may be better to have a false alarm than to miss a fire, in the interest of public safety. False alarms may become impractical for routine monitoring. This trade-off should be decided together with domain experts.

7.9 Why the Model May Focus on Color

The wildfire pictures usually have parts in orange, red, gray, black and white because of flames, smoke, ash and burned vegetation. A CNN can learn these patterns of colour . But colour itself is not reliable. Red soil, artificial lights, autumn vegetation and sunset can produce similar colors. A good wildfire model should learn color, shape, texture, smoke structure and scene context. More diverse training data and targeted augmentation could be helpful to reduce color bias.

7.10 Limitations of the Dataset

The dataset is useful for project development, but has limitations. It has 2,700 images, which is not enough to represent every wildfire condition. It may not include all camera types, all geographic regions, all weather conditions, all times of day, or all stages of fire. A model trained on this data set might behave differently on drone images, satellite images, CCTV footage, thermal images, or real time video frames. The dataset also suffers from class imbalance since there are more no-fire images than fire images.

7.11 Limitations of the Models

The custom CNN is relatively simple and might not learn rich enough features. The MobileNetV2 model is better but the base was frozen in the experiment. Fine-tuning the later layers of MobileNetV2 could improve performance on a specific task. The experiment also used a single decision threshold and did not optimize hyperparameters. More advanced training strategies could involve learning rate scheduling, early stopping, class weights, focal loss, threshold optimization and cross-validation.

7.12 Deployment Considerations

A real wildfire detection system would need more than a trained classifier. Would need a data ingestion pipeline, camera/image source integration, alert thresholds, monitoring dashboard, logging, human review, fail-safe mechanisms, periodic retraining. The model is to be evaluated under real operating conditions. It should also be calibrated so that the probability scores are meaningful. Deployment must include a human-in-the-loop review process, especially given the serious consequences of wildfire alerts.

7.13 Ethical and Social Considerations

Detection of wildfires helps to protect the public and the environment. But automated systems must be used responsibly. It might delay a response or cause unnecessary panic. The system is not intended to be a substitute for trained emergency personnel. It should be framed as a

decision support tool. Sources of data must be obtained legally and ethically and any camera-based monitoring system must respect privacy and local regulations.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The project created a wildfire image classification system using a custom CNN and transfer learning with MobileNetV2. The dataset consisted of 2,700 RGB images belonging to fire and no-fire classes. The images were resized to 224×224 , normalized and loaded using Keras image generators. The custom CNN consisted of convolutional, pooling, dense and dropout layers. The MobileNetV2 model was trained on a new binary classification head with pretrained ImageNet weights.

The custom CNN achieved 82.20

In summary, the project demonstrates that CNNs and transfer learning can be used to classify wildfire images. The system can be used as a prototype and educational project for machine learning. With further data, tuning, validation and deployment engineering such systems could support disaster management and emergency response workflows.

8.2 Future Work

Future work can improve the project in several directions:

1. **Fine-tuning MobileNetV2:** unfreeze selected later layers and train with a small learning rate.
2. **Threshold optimization:** choose a threshold that improves fire recall for emergency detection.
3. **Class weighting:** apply class weights or focal loss to reduce the effect of class imbalance.
4. **More diverse data:** include images from different regions, seasons, camera types, and lighting conditions.
5. **Video-based detection:** extend the model from still images to video streams.
6. **Additional models:** compare EfficientNet, ResNet, DenseNet, and Vision Transformers.
7. **Calibration:** evaluate whether predicted probabilities are well calibrated.
8. **Web interface:** deploy a simple front end using HuggingFace Spaces, Streamlit, Flask, or Vercel.

9. **Human-in-the-loop design:** allow emergency reviewers to confirm or reject model predictions.
10. **Expanded explainability:** generate Grad-CAM heatmaps for multiple correct and incorrect cases.

8.3 Final Statement

The project successfully completes the main objectives of Phase III: documentation of problem, dataset, methodology, experimental setup, results, discussion, conclusion and references. The report can be further edited in Overleaf after feedback. The same content can be converted into an 8 minute slide presentation and 2 minute optional model demo.

References

Bibliography

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Advances in Neural Information Processing Systems*, 2012.
- [2] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520, 2018.
- [4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 618–626, 2017.
- [5] C. Shorten and T. M. Khoshgoftaar, “A Survey on Image Data Augmentation for Deep Learning,” *Journal of Big Data*, vol. 6, article 60, 2019.
- [6] S. J. Pan and Q. Yang, “A Survey on Transfer Learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- [8] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,” 2015. Available: <https://www.tensorflow.org>
- [9] I. El-Madafri, M. Peña, and N. Olmedo-Torre, “The Wildfire Dataset: Enhancing Deep Learning-Based Forest Fire Detection with a Diverse Evolving Open-Source Dataset Focused on Data Representativeness and a Novel Multi-Task Learning Approach,” *Forests*, vol. 14, no. 9, article 1697, 2023.
- [10] I. El-Madafri, “The Wildfire Dataset,” Kaggle. Available: <https://www.kaggle.com/datasets/elmadafri/the-wildfire-dataset>

Appendix A

Important Code Snippets from the Implementation

A.1 Dataset Loading and Generator Setup

The following code snippet shows the main dataset path definitions and image generator configuration used in the notebook.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_path = "/kaggle/input/datasets/elmadafri/the-wildfire-dataset/
    the_wildfire_dataset_2n_version/train"
val_path = "/kaggle/input/datasets/elmadafri/the-wildfire-dataset/
    the_wildfire_dataset_2n_version/val"
test_path = "/kaggle/input/datasets/elmadafri/the-wildfire-dataset/
    the_wildfire_dataset_2n_version/test"

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    shear_range=0.2,
    horizontal_flip=True
)

test_datagen = ImageDataGenerator(rescale=1./255)

train_data = train_datagen.flow_from_directory(
    train_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

val_data = test_datagen.flow_from_directory(
    val_path,
```

```

        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='binary'
    )

test_data = test_datagen.flow_from_directory(
    test_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False
)

```

A.2 Custom CNN Model

The following code snippet shows the custom CNN baseline model.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=15
)

```

A.3 MobileNetV2 Transfer Learning Model

The following code snippet shows the transfer learning model using MobileNetV2.

```

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, GlobalAveragePooling2D, Dense

base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

base_model.trainable = False

inputs = Input(shape=(224,224,3))
x = base_model(inputs)
x = GlobalAveragePooling2D()(x)
outputs = Dense(1, activation='sigmoid')(x)

mobilenet_model = Model(inputs, outputs)

mobilenet_model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

```

A.4 Confusion Matrix and Classification Report

The following code snippet summarizes the evaluation logic.

```

from sklearn.metrics import confusion_matrix, classification_report

predictions = mobilenet_model.predict(test_data)
y_pred = (predictions > 0.5).astype(int).flatten()
y_true = test_data.classes

cm = confusion_matrix(y_true, y_pred)
print(cm)
print(classification_report(y_true, y_pred))

```

A.5 ROC Curve and AUC

```

from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

y_true = test_data.classes
y_scores = mobilenet_model.predict(test_data).ravel()

fpr, tpr, thresholds = roc_curve(y_true, y_scores)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8,6))

```

```
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.3f}')
plt.plot([0,1], [0,1], '--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.savefig('roc_curve.png')
plt.show()
```

A.6 Grad-CAM Heatmap

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

last_conv_layer_name = "Conv_1"
last_conv_layer = base_model.get_layer(last_conv_layer_name)

grad_model = tf.keras.models.Model(
    inputs=base_model.input,
    outputs=[last_conv_layer.output, base_model.output]
)

with tf.GradientTape() as tape:
    conv_outputs, predictions = grad_model(img_array)
    loss = tf.reduce_mean(predictions)

grads = tape.gradient(loss, conv_outputs)
pooled_grads = tf.reduce_mean(grads, axis=(0,1,2))
conv_outputs = conv_outputs[0]

heatmap = tf.reduce_sum(pooled_grads * conv_outputs, axis=-1)
heatmap = np.maximum(heatmap, 0)
heatmap = heatmap / np.max(heatmap)

plt.figure(figsize=(6,6))
plt.imshow(heatmap, cmap='jet')
plt.colorbar()
plt.title("Grad-CAM Heatmap")
plt.savefig("gradcam_heatmap.png")
plt.show()
```